

```

// types/overlay.ts
export type OverlayPosition = 'top-left' | 'top-right' | 'bottom-left' | 'bottom-right';
export type OverlayEntryAnimation = 'slide-up' | 'slide-in-left' | 'fade-in' | 'typewriter';
export type OverlaySpecialEffectType = 'sparkle' | 'confetti' | 'glow' | 'none';
export type OverlayColorPreset = 'darkTurquoise' | 'black' | 'transparent';

export type OverlayFont =
  | 'Open Sans'
  | 'Montserrat'
  | 'Comic Neue'
  | 'Inter'
  | 'System UI';

export interface OverlaySettings {
  position: OverlayPosition;
  opacity: number; // 0.3–1.0
  bgColor: string | OverlayColorPreset; // HEX lub preset
  textColor: string | 'white' | 'gold' | 'turquoise';
  durationSec: number; // 5–15
  fontFamily: OverlayFont;
  entryAnimation: OverlayEntryAnimation;
  specialEffectThreshold: number; // w USDC
  specialEffectType: OverlaySpecialEffectType;
  soundEffectUrl?: string;
}

export const defaultOverlaySettings: OverlaySettings = {
  position: 'top-right',
  opacity: 0.85,
  bgColor: 'darkTurquoise', // lub np. '#07393a'
  textColor: 'white',
  durationSec: 8,
  fontFamily: 'Inter',
  entryAnimation: 'slide-up',
  specialEffectThreshold: 50,
  specialEffectType: 'confetti',
  soundEffectUrl: undefined,
};

export function sanitizeOverlaySettings(s: OverlaySettings): OverlaySettings {
  const clamp = (n: number, min: number, max: number) => Math.min(Math.max(n, min), max);
  return {
    ...s,
    opacity: clamp(s.opacity, 0.3, 1.0),
  };
}

```

```

    durationSec: Math.round(clamp(s.durationSec, 5, 15)),
    specialEffectThreshold: s.specialEffectThreshold < 0 ? 0 : s.specialEffectThreshold,
  };
}

```

ts

Skopiuuj kod

// stores/overlaySettingsStore.ts

```

import { create } from 'zustand';
import { devtools } from 'zustand/middleware';
import {
  OverlaySettings,
  defaultOverlaySettings,
  sanitizeOverlaySettings,
} from '@types/overlay';

```

```

type OverlayState = {
  settings: OverlaySettings;
  pending: boolean;
  error?: string;
  set: (patch: Partial<OverlaySettings>) => void;
  reset: () => void;
  load: (creatorId: string, token?: string) => Promise<void>;
  save: (creatorId: string, token?: string) => Promise<void>;
};

```

```

export const useOverlaySettingsStore = create<OverlayState>()(
  devtools((set, get) => ({
    settings: defaultOverlaySettings,
    pending: false,
    error: undefined,

    set: (patch) =>
      set((state) => ({
        settings: sanitizeOverlaySettings({ ...state.settings, ...patch }),
      })),

    reset: () => set({ settings: defaultOverlaySettings, error: undefined }),

    load: async (creatorId, token) => {
      set({ pending: true, error: undefined });
      try {
        const res = await fetch(`/api/overlay/settings/${creatorId}`, {
          headers: {
            'Content-Type': 'application/json',

```

```

    ...(token ? { Authorization: `Bearer ${token}` } : {}),
  },
  cache: 'no-store',
});

if (res.status === 404) {
  set({ settings: defaultOverlaySettings, pending: false });
  return;
}
if (!res.ok) {
  const msg = `LOAD_${res.status}`;
  set({ error: msg, pending: false });
  return;
}

const data = (await res.json()) as OverlaySettings;
set({ settings: sanitizeOverlaySettings(data), pending: false });
} catch (e) {
  set({ error: 'LOAD_ERROR', pending: false });
}
},

save: async (creatorId, token) => {
  const body = JSON.stringify(sanitizeOverlaySettings(get().settings));
  set({ pending: true, error: undefined });
  try {
    const res = await fetch(`/api/overlay/settings/${creatorId}`, {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
        ...(token ? { Authorization: `Bearer ${token}` } : {}),
      },
      body,
    });
    if (!res.ok) {
      const msg = `SAVE_${res.status}`;
      set({ error: msg, pending: false });
      return;
    }
    set({ pending: false });
  } catch {
    set({ error: 'SAVE_ERROR', pending: false });
  }
},

```

```
    )))  
  );
```

OverlayEditor.tsx — gotowe.

```
// components/overlay/OverlayEditor.tsx  
'use client';  
  
import { useEffect, useMemo, useRef, useState } from 'react';  
import { useOverlaySettingsStore } from  
'@/stores/overlaySettingsStore';  
import type {  
  OverlaySettings,  
  OverlayPosition,  
  OverlayEntryAnimation,  
  OverlaySpecialEffectType,  
  OverlayColorPreset,  
  OverlayFont,  
} from '@/types/overlay';  
  
type Props = { creatorId: string; token?: string };  
  
const POS: OverlayPosition[] = ['top-left', 'top-right',  
  'bottom-left', 'bottom-right'];  
const ANIM: OverlayEntryAnimation[] = ['slide-up', 'slide-in-left',  
  'fade-in', 'typewriter'];  
const FX: OverlaySpecialEffectType[] = ['sparkle', 'confetti', 'glow',  
  'none'];  
const PRESET_BG: Record<OverlayColorPreset, string> = {  
  darkTurquoise: '#07393a',  
  black: '#000000',  
  transparent: 'rgba(0,0,0,0)',  
};  
const FONT: OverlayFont[] = ['Inter', 'Open Sans', 'Montserrat',  
  'Comic Neue', 'System UI'];  
  
export default function OverlayEditor({ creatorId, token }: Props) {
```

```

    const { settings, set, load, save, reset, pending, error } =
useOverlaySettingsStore();
    const [previewKey, setPreviewKey] = useState(0); // restart animacji
    const [nowFx, setNowFx] =
useState<OverlaySpecialEffectType>('none');

    useEffect(() => {
        void load(creatorId, token);
    }, [creatorId, token, load]);

    const bg = useMemo(() => {
        if (settings.bgColor === 'darkTurquoise' || settings.bgColor ===
'black' || settings.bgColor === 'transparent') {
            return PRESET_BG[settings.bgColor];
        }
        return settings.bgColor; // HEX
    }, [settings.bgColor]);

    const textColor = useMemo(() => {
        if (settings.textColor === 'white') return '#ffffff';
        if (settings.textColor === 'gold') return '#FFD700';
        if (settings.textColor === 'turquoise') return '#6FE7E7';
        return settings.textColor; // HEX
    }, [settings.textColor]);

    const posClass = useMemo(() => {
        const base = 'absolute m-4';
        switch (settings.position) {
            case 'top-left':
                return `${base} left-0 top-0`;
            case 'top-right':
                return `${base} right-0 top-0`;
            case 'bottom-left':
                return `${base} left-0 bottom-0`;
            case 'bottom-right':
                return `${base} right-0 bottom-0`;
        }
    }, [settings.position]);

```

```

const animClass = useMemo(() => {
  switch (settings.entryAnimation) {
    case 'slide-up':
      return 'oi-slide-up';
    case 'slide-in-left':
      return 'oi-slide-left';
    case 'fade-in':
      return 'oi-fade-in';
    case 'typewriter':
      return 'oi-typewriter';
  }
}, [settings.entryAnimation]);

const triggerPreview = () => {
  setPreviewKey((k) => k + 1);
  setNowFx(settings.specialEffectType);
  // auto-clear FX po 1.5s
  window.setTimeout(() => setNowFx('none'), 1500);
};

const demoTip = { name: 'alice.eth', amount: 12.34, currency:
'USDC', msg: 'Love your stream!' };

return (
  <div className="grid grid-cols-1 gap-6 md:grid-cols-2">
    { /* LIVE PREVIEW */ }
    <div className="rounded-[16px] border border-white/10 bg-card
p-4">
      <div className="mb-3 flex items-center justify-between">
        <h3 className="text-base font-semibold">Live Preview</h3>
        <div className="flex gap-2">
          <button
            onClick={triggerPreview}
            className="rounded-[12px] bg-[#FFD700] px-3 py-1.5
text-sm font-semibold text-[#003737] transition
hover:-translate-y-[1px]"
          >

```

```

        Trigger test tip
      </button>
      <button
        onClick={() => reset()}
        className="rounded-[12px] border border-white/15 px-3
py-1.5 text-sm text-[#DDE0DA] hover:-translate-y-[1px] transition"
      >
        Reset
      </button>
    </div>
  </div>

  <div className="relative h-[360px] w-full overflow-hidden
rounded-[12px] bg-black/30">
    {/* symulacja sceny */}
    <div className="pointer-events-none absolute inset-0
bg-[radial-gradient(ellipse_at_30%_20%, rgba(255,215,0,0.06), transparen
t_40%), radial-gradient(ellipse_at_70%_80%, rgba(0,255,255,0.06), transpa
rent_40%)]" />
      {/* overlay */}
      <OverlayBox
        key={previewKey}
        className={posClass}
        style={{ opacity: settings.opacity, background: bg, color:
textColor, fontFamily: settings.fontFamily }}
        animClass={animClass}
        durationSec={settings.durationSec}
        typewriter={settings.entryAnimation === 'typewriter'}
        data={demoTip}
      />
      {nowFx !== 'none' ? <Fx fx={nowFx} /> : null}
    </div>

    <div className="mt-3 text-xs text-[#BCC1B6]">
      Preview uses current settings. Position reflects selected
corner. Animation restarts on "Trigger test tip".
    </div>
  </div>

```

```

    { /* CONTROLS */ }
    <form
      className="rounded-[16px] border border-white/10 bg-card p-4
md:p-6 space-y-4"
      onSubmit={(e) => {
        e.preventDefault();
        void save(creatorId, token);
      }}
    >
      <h3 className="text-base font-semibold">Overlay settings</h3>

      { /* position */ }
      <Field label="Position">
        <select
          value={settings.position}
          onChange={(e) => set({ position: e.target.value as
OverlayPosition })}
          className="w-full rounded-[10px] border border-white/15
bg-transparent px-3 py-2 text-sm"
        >
          {POS.map((p) => (
            <option key={p} value={p} className="bg-[#042627]">
              {p}
            </option>
          ))}
        </select>
      </Field>

      { /* opacity */ }
      <Field label={`Opacity: ${settings.opacity.toFixed(2)}%`}>
        <input
          type="range"
          min={0.3}
          max={1}
          step={0.01}
          value={settings.opacity}
          onChange={(e) => set({ opacity: Number(e.target.value) })}
        >

```



```

        className="w-full"
      />
    </Field>

    { /* bgColor */ }
    <Field label="Background">
      <div className="grid grid-cols-2 gap-2">
        <select
          value={['darkTurquoise', 'black',
'transparent'].includes(String(settings.bgColor)) ?
String(settings.bgColor) : 'custom' }
          onChange={(e) => {
            const v = e.target.value as OverlayColorPreset |
'custom';
            set({ bgColor: v === 'custom' ? '#07393a' : v });
          }}
          className="rounded-[10px] border border-white/15
bg-transparent px-3 py-2 text-sm"
        >
          <option value="darkTurquoise"
className="bg-[#042627]">darkTurquoise</option>
          <option value="black"
className="bg-[#042627]">black</option>
          <option value="transparent"
className="bg-[#042627]">transparent</option>
          <option value="custom" className="bg-[#042627]">custom
HEX</option>
        </select>
        <input
          type="color"
          value={String(['darkTurquoise', 'black',
'transparent'].includes(String(settings.bgColor)) ?
PRESET_BG[settings.bgColor as OverlayColorPreset] : settings.bgColor)}
          onChange={(e) => set({ bgColor: e.target.value })}
          className="h-10 w-full cursor-pointer rounded-[10px]
border border-white/15 bg-transparent"
        />
      </div>

```

```

</Field>

{/* textColor */}
<Field label="Text color">
  <div className="grid grid-cols-2 gap-2">
    <select
      value={['white', 'gold',
'turquoise'].includes(String(settings.textColor)) ?
String(settings.textColor) : 'custom'}
      onChange={(e) => {
        const v = e.target.value as 'white' | 'gold' |
'turquoise' | 'custom';
        set({ textColor: v === 'custom' ? '#DDE0DA' : v });
      }}
      className="rounded-[10px] border border-white/15
bg-transparent px-3 py-2 text-sm"
    >
      <option value="white"
className="bg-[#042627]">white</option>
      <option value="gold"
className="bg-[#042627]">gold</option>
      <option value="turquoise"
className="bg-[#042627]">turquoise</option>
      <option value="custom" className="bg-[#042627]">custom
HEX</option>
    </select>
    <input
      type="color"
      value={String(['white', 'gold',
'turquoise'].includes(String(settings.textColor)) ?
(settings.textColor === 'white' ? '#ffffff' : settings.textColor ===
'gold' ? '#FFD700' : '#6FE7E7') : settings.textColor)}
      onChange={(e) => set({ textColor: e.target.value })}
      className="h-10 w-full cursor-pointer rounded-[10px]
border border-white/15 bg-transparent"
    />
  </div>
</Field>

```

```

    { /* duration */
    <Field label={`Display time: ${settings.durationSec}s`} >
      <input
        type="range"
        min={5}
        max={15}
        step={1}
        value={settings.durationSec}
        onChange={(e) => set({ durationSec: Number(e.target.value)
    }}}

    className="w-full"
  />
</Field>

{ /* font */
<Field label="Font">
  <select
    value={settings.fontFamily}
    onChange={(e) => set({ fontFamily: e.target.value as
typeof settings.fontFamily })}
    className="w-full rounded-[10px] border border-white/15
bg-transparent px-3 py-2 text-sm"
    >
      {FONT.map((f) => (
        <option key={f} value={f} className="bg-[#042627]">
          {f}
        </option>
      ))}
    </select>
  </Field>

{ /* entry animation */
<Field label="Entry animation">
  <select
    value={settings.entryAnimation}
    onChange={(e) => set({ entryAnimation: e.target.value as
OverlayEntryAnimation })}

```

```

        className="w-full rounded-[10px] border border-white/15
bg-transparent px-3 py-2 text-sm"
      >
        {ANIM.map((a) => (
          <option key={a} value={a} className="bg-[#042627]">
            {a}
          </option>
        ))}
      </select>
    </Field>

    {/* special effect */}
    <Field label="Special effect on big tips">
      <div className="grid grid-cols-2 gap-2">
        <select
          value={settings.specialEffectType}
          onChange={(e) => set({ specialEffectType: e.target.value
as OverlaySpecialEffectType })}
          className="rounded-[10px] border border-white/15
bg-transparent px-3 py-2 text-sm"
        >
          {FX.map((f) => (
            <option key={f} value={f} className="bg-[#042627]">
              {f}
            </option>
          ))}
        </select>
        <input
          type="number"
          min={0}
          step={1}
          value={settings.specialEffectThreshold}
          onChange={(e) => set({ specialEffectThreshold:
Number(e.target.value) })}
          className="rounded-[10px] border border-white/15
bg-transparent px-3 py-2 text-sm"
          placeholder="Threshold in USDC"
        />

```

```

    </div>
  </Field>

  {/* sound */}
  <Field label="Sound effect URL (optional)">
    <input
      type="url"
      value={settings.soundEffectUrl ?? ''}
      onChange={(e) => set({ soundEffectUrl: e.target.value ||
undefined })}
      className="w-full rounded-[10px] border border-white/15
bg-transparent px-3 py-2 text-sm"
      placeholder="https://cdn.your-bucket/sound.mp3"
    />
  </Field>

  <div className="pt-2 flex items-center gap-3">
    <button
      type="submit"
      disabled={pending}
      className="rounded-[12px] bg-[#FFD700] px-4 py-2 text-sm
font-semibold text-[#003737] disabled:opacity-60"
    >
      {pending ? 'Saving...' : 'Save settings'}
    </button>
    {error ? <span className="text-sm text-red-400">{error}</span> : null}
  </div>
</form>

{/* Animacje i FX (scoped) */}
<style jsx>{`
  .oi-fade-in {
    animation: oifade 200ms ease-out both;
  }
  .oi-slide-up {
    animation: oislideup 220ms ease-out both;
  }
`}
```

```

    .oi-slide-left {
        animation: oislideleft 220ms ease-out both;
    }
    .oi-typewriter .tw {
        overflow: hidden;
        white-space: nowrap;
        border-right: 2px solid currentColor;
        animation: oitw 1s steps(20) 0s 1 both, oicaret 750ms
step-end infinite;
    }
    @keyframes oifade {
        from { opacity: 0; transform: translateY(4px); }
        to { opacity: 1; transform: translateY(0); }
    }
    @keyframes oislideup {
        from { opacity: 0; transform: translateY(12px); }
        to { opacity: 1; transform: translateY(0); }
    }
    @keyframes oislideleft {
        from { opacity: 0; transform: translateX(-12px); }
        to { opacity: 1; transform: translateX(0); }
    }
    @keyframes oitw {
        from { width: 0 }
        to { width: 100% }
    }
    @keyframes oicaret {
        50% { border-color: transparent; }
    }
    /* FX */
    .fx-confetti { animation: fxpop 600ms ease-out both; filter:
drop-shadow(0 0 6px rgba(255,215,0,0.6)); }
    .fx-sparkle { animation: fxspark 900ms ease-out both; }
    .fx-glow { animation: fxglow 1200ms ease-out both; }
    @keyframes fxpop { from { transform: scale(0.9); opacity: .0
} to { transform: scale(1); opacity: 1 } }
    @keyframes fxspark { 0% { opacity: 0 } 20% { opacity: 1 } 100%
{ opacity: 0 } }

```

```

        @keyframes fxglow { 0% { box-shadow: 0 0 0 rgba(255,215,0,0)
    } 50% { box-shadow: 0 0 24px rgba(255,215,0,0.55) } 100% { box-shadow:
    0 0 0 rgba(255,215,0,0) } }
        `}</style>
    </div>
    );
}

```

```

function Field(props: { label: string; children: React.ReactNode }) {
    return (
        <label className="block">
            <div className="mb-1 text-sm text-[#DDE0DA]">{props.label}</div>
            {props.children}
        </label>
    );
}

```

```

function OverlayBox(props: {
    className: string;
    style: React.CSSProperties;
    animClass: string;
    durationSec: number;
    typewriter: boolean;
    data: { name: string; amount: number; currency: string; msg: string
};
}) {
    const { className, style, animClass, durationSec, typewriter, data }
= props;
    const [visible, setVisible] = useState(true);
    const audioRef = useRef<HTMLAudioElement | null>(null);

    useEffect(() => {
        setVisible(true);
        const t = window.setTimeout(() => setVisible(false), durationSec *
1000);
        return () => window.clearTimeout(t);
    }, [durationSec]);
}

```

```

return (
  <>
    {visible ? (
      <div className={` ${className} ${animClass}`} style={{
...style, borderRadius: 14, padding: '12px 14px' }}>
      <div className="flex items-start gap-3">
        <div
          className="h-10 w-10 shrink-0 rounded-full"
          style={{ background: 'linear-gradient(135deg,#FFD700,
#A27A00)' }}
          aria-hidden
        />
        <div className="min-w-[220px] max-w-[360px]">
          <div className="text-sm font-semibold leading-tight"
style={{ color: style.color }}>
            {data.name} tipped {data.amount} {data.currency}
          </div>
          <div className={`mt-0.5 text-sm ${typewriter ? 'tw' :
''}`} style={{ color: style.color }}>
            {data.msg}
          </div>
        </div>
      </div>
    ) : null}
    {audioRef.current ? null : null}
  </>
);
}

```

```

function Fx({ fx }: { fx: OverlaySpecialEffectType }) {
  if (fx === 'confetti') return <div className="pointer-events-none
absolute inset-0 fx-confetti" />;
  if (fx === 'sparkle')
    return (
      <div className="pointer-events-none absolute inset-0
fx-sparkle">

```



```

        <div className="absolute left-1/4 top-1/3 h-2 w-2 rounded-full
bg-[#FFD700]" />
        <div className="absolute left-2/3 top-2/3 h-1.5 w-1.5
rounded-full bg-[#6FE7E7]" />
      </div>
    );
    if (fx === 'glow') return <div className="pointer-events-none
absolute inset-0 fx-glow rounded-[12px]" />;
    return null;
  }
}

```

```

// app/creator-studio/page.tsx
'use client';

```

```

import { useState, type ReactNode } from 'react';

```

```

type Tab = 'QR' | 'Overlay' | 'Widget' | 'TipModal' | 'Presets' | 'Profile' | 'Learn';

```

```

const TABS: Tab[] = ['QR', 'Overlay', 'Widget', 'TipModal', 'Presets', 'Profile', 'Learn'];

```

```

export default function CreatorStudioPage() {
  const [active, setActive] = useState<Tab>('QR');

```

```

  return (
    <main className="mx-auto max-w-[1480px] px-4 py-8 text-[#DDE0DA]">
      <h1 className="mb-2 text-2xl md:text-3xl font-semibold">Creator Studio</h1>

```

```

      {/* Tabs */}
      <nav className="mb-6 flex flex-wrap gap-2">
        {TABS.map((t) => (
          <button
            key={t}
            onClick={() => setActive(t)}
            className={[
              'rounded-[12px] px-4 py-2 text-sm transition',
              active === t
                ? 'bg-[#FFD700] text-[#003737] font-semibold'
                : 'border border-white/10 bg-card text-[#DDE0DA] hover:-translate-y-[1px]',
            ].join(' ')}
          >
            {t}
          </button>
        ))}

```

```

    </nav>

    { /* Panel */
    <section className="rounded-[16px] border border-white/10 bg-card p-6 md:p-8">
      <TabPanel tab={active} />
    </section>
  </main>
);
}

```

```

function TabPanel({ tab }: { tab: Tab }) {
  const map: Record<Tab, ReactNode> = {
    QR: <QRSection />,
    Overlay: <OverlaySection />,
    Widget: <WidgetSection />,
    TipModal: <TipModalSection />,
    Presets: <PresetsSection />,
    Profile: <ProfileSection />,
    Learn: <LearnSection />,
  };
  return <>{map[tab]}</>;
}

```

/* ===== Sekcje — placeholderzy do wypełnienia w kolejnych krokach ===== */

```

function QRSection() {
  return (
    <div>
      <h2 className="mb-3 text-base font-semibold">QR</h2>
      <p className="text-[14px] leading-[1.5] text-[#BCC1B6]">
        Generator QR do profilu + podgląd. (Implementacja w kroku „next: QR”)
      </p>
    </div>
  );
}

```

```

function OverlaySection() {
  return (
    <div>
      <h2 className="mb-3 text-base font-semibold">Overlay</h2>
      <p className="text-[14px] leading-[1.5] text-[#BCC1B6]">
        Edytor nakładki live + preview/iframe. (Implementacja w kroku „next: Overlay”)
      </p>
    </div>
  );
}

```

```
);  
}
```

```
function WidgetSection() {  
  return (  
    <div>  
      <h2 className="mb-3 text-base font-semibold">Widget</h2>  
      <p className="text-[14px] leading-[1.5] text-[#BCC1B6]">  
        Standardowy widget i konfiguracja. (Implementacja w kroku „next: Widget”)  
      </p>  
    </div>  
  );  
}
```

```
function TipModalSection() {  
  return (  
    <div>  
      <h2 className="mb-3 text-base font-semibold">TipModal</h2>  
      <p className="text-[14px] leading-[1.5] text-[#BCC1B6]">  
        Modal napiwków z podglądem. (Implementacja w kroku „next: TipModal”)  
      </p>  
    </div>  
  );  
}
```

```
function PresetsSection() {  
  return (  
    <div>  
      <h2 className="mb-3 text-base font-semibold">Presets</h2>  
      <p className="text-[14px] leading-[1.5] text-[#BCC1B6]">  
        Gotowe warianty „na klik” dla Overlay/Widget/TipModal. (Implementacja w kroku „next:  
Presets”)  
      </p>  
    </div>  
  );  
}
```

```
function ProfileSection() {  
  return (  
    <div>  
      <h2 className="mb-3 text-base font-semibold">Profile</h2>  
      <p className="text-[14px] leading-[1.5] text-[#BCC1B6]">  
        Pasek postępu celu i ściana fanów. (Implementacja w kroku „next: Profile”)  
      </p>  
    </div>  
  );  
}
```

```
    </div>
  );
}
```

```
function LearnSection() {
  return (
    <div>
      <h2 className="mb-3 text-base font-semibold">Learn</h2>
      <p className="text-[14px] leading-[1.5] text-[#BCC1B6]">
        Przewodnik krok-po-kroku + asystent AI. (Implementacja w kroku „next: Learn”)
      </p>
    </div>
  );
}
```

```
// app/onboarding/page.tsx
'use client';
```

```
import { useState, useMemo, FormEvent } from 'react';
import Image from 'next/image';
import { useRouter } from 'next/navigation';
```

```
type Industry =
  | 'Hospitality'
  | 'Beauty & Wellness'
  | 'Music'
  | 'Streaming & Gaming'
  | 'Education'
  | 'Street Art'
  | 'Other';
```

```
const INDUSTRIES: Industry[] = [
  'Streaming & Gaming',
  'Music',
  'Education',
  'Hospitality',
  'Beauty & Wellness',
  'Street Art',
  'Other',
];
```

```
const SOCIALS = [
  { key: 'x', label: 'X (Twitter)' },
  { key: 'instagram', label: 'Instagram' },
];
```

```

    { key: 'youtube', label: 'YouTube' },
    { key: 'twitch', label: 'Twitch' },
    { key: 'discord', label: 'Discord' },
    { key: 'telegram', label: 'Telegram' },
  ] as const;

export default function OnboardingPage() {
  const router = useRouter();

  // 1) Branża
  const [industry, setIndustry] = useState<Industry>('Streaming & Gaming'); // domyślne
  const [skipIndustry, setSkipIndustry] = useState(false);

  // 2) Social
  const [connected, setConnected] = useState<Record<(typeof SOCIALS)[number]['key'],
boolean>>({
    x: false,
    instagram: false,
    youtube: false,
    twitch: false,
    discord: false,
    telegram: false,
  });

  // 3) Avatar
  const [file, setFile] = useState<File | null>(null);
  const previewUrl = useMemo(() => (file ? URL.createObjectURL(file) : ''), [file]);

  async function onSubmit(e: FormEvent) {
    e.preventDefault();

    // [TODO] Zapis branży
    // POST /api/profile/industry { industry } — pomiń jeśli skipIndustry === true
    try {
      if (!skipIndustry) {
        await fetch('/api/profile/industry', {
          method: 'POST',
          headers: { 'Content-Type': 'application/json' },
          body: JSON.stringify({ industry }),
        });
      }
    } catch {}

    // [TODO] Social OAuth — faktyczne podpięcie dzieje się poza tym formularzem.
  }
}

```

// Tu tylko hint: wysyłamy stan "chce podłączyć" do backendu, by pokazać kolejne kroki w profilu.

```
try {
  await fetch('/api/profile/socials/intent', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ requested: Object.entries(connected).filter(([, v]) => v).map(([k]) => k) })),
  });
} catch {}
```

// [TODO] Upload avatara

```
try {
  if (file) {
    const fd = new FormData();
    fd.append('avatar', file);
    await fetch('/api/profile/avatar', { method: 'POST', body: fd });
  }
} catch {}
```

// Dalej: profil pusty -> /creator-studio lub /profile

```
router.push('/creator-studio');
}
```

return (

```
<main className="mx-auto max-w-[980px] px-4 py-8 text-[#DDE0DA]">
  <header className="mb-6">
    <h1 className="text-2xl md:text-3xl font-semibold">Let's set up your profile</h1>
    <p className="mt-1 text-[14px] leading-[1.5] text-[#BCC1B6]">
      Te odpowiedzi pomogą w wyszukiwaniu i trafnych rekomendacjach. Możesz pominąć i
      wypełnić później.
    </p>
  </header>
```

```
<form onSubmit={onSubmit} className="space-y-8">
  { /* 1) Branża */ }
  <section className="rounded-[16px] border border-white/10 bg-card p-6 md:p-8">
    <div className="flex items-center justify-between">
      <h2 className="text-base font-semibold leading-[1.5]">Your industry</h2>
      <label className="flex items-center gap-2 text-xs text-[#BCC1B6]">
        <input
          type="checkbox"
          checked={skipIndustry}
          onChange={(e) => setSkipIndustry(e.target.checked)}
        </input>
      </label>
    </div>
  </section>
</form>
```

```

    />
    Skip
  </label>
</div>

```

```

<p className="mt-1 mb-4 text-[13px] text-[#8F9687]">
  Used for discovery and search ranking.
</p>

```

```

<fieldset disabled={skipIndustry} className={skipIndustry ? 'opacity-50
pointer-events-none' : ''}>
  <select
    value={industry}
    onChange={(e) => setIndustry(e.target.value as Industry)}
    className="w-full rounded-[12px] border border-white/10 bg-transparent px-3 py-2
text-sm"
  >
    {INDUSTRIES.map((opt) => (
      <option key={opt} value={opt} className="bg-[#042627]">
        {opt}
      </option>
    ))}
  </select>
</fieldset>
</section>

```

```

{ /* 2) Social accounts */ }
<section className="rounded-[16px] border border-white/10 bg-card p-6 md:p-8">
  <h2 className="text-base font-semibold leading-[1.5]">Connect socials</h2>
  <p className="mt-1 mb-4 text-[13px] text-[#8F9687]">Add at least one for better reach.
You can do it later.</p>

```

```

<ul className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 gap-3">
  {SOCIALS.map((s) => (
    <li key={s.key}>
      <button
        type="button"
        onClick={() =>
          setConnected((prev) => ({ ...prev, [s.key]: !prev[s.key] }))
        }
        className={[
          'w-full rounded-[12px] border px-3 py-2 text-sm text-left transition',
          connected[s.key]
            ? 'border-[#FFD700] bg-white/5'

```

```

      : 'border-white/10 bg-transparent hover:bg-white/5',
    ].join(' ')}
  >
    <span className="mr-2 inline-block h-2 w-2 rounded-full align-middle"
      style={{ background: connected[s.key] ? '#FFD700' : '#56605A' }} />
    {s.label}
    <span className="float-right text-xs text-[#BCC1B6]">
      {connected[s.key] ? 'selected' : 'select'}
    </span>
  </button>
</li>
  )})
</ul>

<div className="mt-3 text-[12px] text-[#8F9687]">
  OAuth otworzy nowe okno, gdy wybierzesz „Continue”. Nie publikujemy bez Twojej
zgody.
</div>
</section>

{/* 3) Avatar */}
<section className="rounded-[16px] border border-white/10 bg-card p-6 md:p-8">
  <h2 className="text-base font-semibold leading-[1.5]">Profile photo</h2>
  <p className="mt-1 mb-4 text-[13px] text-[#8F9687]">PNG/JPG, min 512×512. Możesz
pomiąć.</p>

  <div className="flex items-center gap-4">
    <div className="h-20 w-20 overflow-hidden rounded-full border border-white/10
bg-white/5">
      {previewUrl ? (
        // eslint-disable-next-line @next/next/no-img-element
        <img src={previewUrl} alt="" className="h-full w-full object-cover" />
      ) : (
        <div className="flex h-full w-full items-center justify-center text-xs
text-[#8F9687]">no photo</div>
      )}
    </div>

    <label className="inline-flex items-center gap-2 rounded-[12px] border border-white/10
px-3 py-2 text-sm cursor-pointer hover:bg-white/5">
      <input
        type="file"
        accept="image/*"
        className="hidden"

```



```

        onChange={(e) => setFile(e.target.files?.[0] ?? null)}
      />
      Upload
    </label>

    {file ? (
      <button
        type="button"
        onClick={() => setFile(null)}
        className="rounded-[12px] border border-white/10 px-3 py-2 text-sm
hover:bg-white/5"
      >
        Remove
      </button>
    ) : null}
  </div>
</section>

  {/* CTA */}
  <footer className="flex flex-col-reverse gap-3 sm:flex-row sm:items-center
sm:justify-between">
    <button
      type="button"
      onClick={() => router.push('/creator-studio')}
      className="rounded-[12px] border border-white/10 px-4 py-2 text-sm hover:bg-white/5"
    >
      Skip for now
    </button>

    <button
      type="submit"
      className="rounded-[12px] bg-[#FFD700] px-4 py-2 text-sm font-semibold
text-[#003737] transition hover:-translate-y-[1px]"
    >
      Continue
    </button>
  </footer>
</form>
</main>
);
}

```

1) Zmienna środowiskowa

```
# .env.local
GEMINI_API_KEY=***TU_KLUCZ***
```

2) Endpoint API

```
// app/api/ai/generate-card-content/route.ts
import { NextResponse } from 'next/server';
import { GoogleGenerativeAI } from '@google/generative-ai';

type Profile = {
  username: string;
  profileUrl: string;          // pełny URL profilu
  headline?: string;           // z onboardingu
  about?: string;              // z onboardingu
  focus?: string;              // z onboardingu
  links?: { label: string; url: string }[]; // latest content
  goal?: { label: string; amount: number; currency: 'USDC'|'USD'|'PLN'
};
  industry?: string;           // z kroku „industry”
};

type CardContent = {
  headline: string;            // max ~12 słów
  summary: string;             // 2-3 zdania, EN
};

function extractJson(text: string): unknown {
  // usuń ewentualne ```json ... ``` lub śmieci przed/po
  const fence = text.match(/```json([\s\S]*?)```/i);
  const raw = fence ? fence[1] : text;
  const start = raw.indexOf('{');
  const end = raw.lastIndexOf('}');
  const sliced = start >= 0 && end > start ? raw.slice(start, end + 1)
: raw;
  return JSON.parse(sliced);
}

export async function POST(req: Request) {
```

```

    try {
      const body = await req.json();
      const userProfile: Profile | undefined = body?.userProfile;
      if (!userProfile?.profileUrl || !userProfile?.username) {
        return NextResponse.json({ error: 'Invalid payload' }, { status:
400 });
      }

      const apiKey = process.env.GEMINI_API_KEY;
      if (!apiKey) {
        return NextResponse.json({ error: 'Missing GEMINI_API_KEY' }, {
status: 500 });
      }

      const genAI = new GoogleGenerativeAI(apiKey);
      // Model: użyj stabilnego wariantu dostępnego w projekcie (np.
gemini-pro / 1.5-pro)
      const model = genAI.getGenerativeModel({ model: 'gemini-pro' });

      const prompt = `
You are a personal branding copywriter with a witty, modern tone.
Create short English copy for an A4 promo poster that sends people to
a TipJar profile.

Return ONLY valid JSON with exactly:
{
  "headline": "≤12 words. Catchy. No emojis.",
  "summary": "2–3 sentences. Crisp. Mention what the creator does and
why to support."
}

User data (JSON):
${JSON.stringify(userProfile, null, 2)}
`;

      const result = await model.generateContent(prompt);
      const text = result.response.text();
      const json = extractJson(text) as CardContent;

```

```

    // walidacja minimalna
    if (!json?.headline || !json?.summary) {
        return NextResponse.json({ error: 'Malformed AI response' }, {
status: 502 });
    }

    return NextResponse.json(json, { status: 200 });
} catch (err: any) {
    return NextResponse.json(
        { error: 'Gemini error', detail: String(err?.message ?? err) },
        { status: 500 }
    );
}
}

```

3) Typ dla klienta

```

// types/ai.ts
export type CardContent = { headline: string; summary: string };

```

4) Instalacja pakietu

```

# jeden raz w projekcie
pnpm add @google/generative-ai
# lub: npm i @google/generative-ai

```

Co się dzieje:

- Przyjmuje `UserProfile` z frontu.
- Buduje prompt z danymi Onboardingu i celu.
- Wzywa Gemini i zwraca `{ headline, summary }`.
- JSON parser wycina ewentualne code-fence. Zwraca 400/500 na błędach.